



---

# Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

|                    |                              |
|--------------------|------------------------------|
| NIM                | 71251221                     |
| Nama Lengkap       | RIZKY FEBRIANTO KRISTIANSYAH |
| Minggu ke / Materi | 12 / Tipe Data Dictionary    |

PROGRAM STUDI INFORMATIKA  
FAKULTAS TEKNOLOGI INFORMASI  
UNIVERSITAS KRISTEN DUTA WACANA  
YOGYAKARTA  
2026

## BAGIAN 1: Materi

### Definisi *Dictionary*

Dictionary punya kemiripan dengan tipe data list, tetapi dictionary punya sifat yang lebih umum. Dalam list, index harus berupa integer sedangkan dalam dictionary, index bisa dapat berupa apapun. Dictionary memiliki anggota yang terdiri dari pasangan **key : value**. **Key** harus bersifat unik, yakni tidak boleh ada dua **key** yang sama dalam satu dictionary.

Dictionary dapat diartikan juga sebagai pemetaan antara sekumpulan index (yang disebut sebagai **key**) dan sekumpulan value (yang disebut sebagai **value**). Asosiasi antara key dan value disebut sebagai **item**. Contohnya, dictionary dapat digunakan untuk membuat kamus bahasa Inggris ke Spanyol;, di mana English word menjadi key dan Spanish word menjadi valuenya.

Untuk membuat dictionary baru yang kosong, digunakan built-in function Python yaitu dict(). Karena dict merupakan built-in function, penggunaannya sebagai nama variabel perlu dihindari. Dictionary kosong direpresentasikan dengan tanda kurung kurawal { }. Untuk menambahkan item ke dalam dictionary, digunakan tanda kurung kotak [ ] dengan key yang ditentukan sebagai indeks. Format output dari dictionary juga sekaligus merupakan format input yang valid, sehingga dictionary dapat dibuat langsung dengan menuliskan pasangan key-value di dalam kurung kurawal. Apabila key yang digunakan tidak ditemukan dalam dictionary, Python akan menghasilkan error berupa KeyError.

```
>>> eng2sp = {"one" : "uno", "two" : "dos", "three" : "tres"}
>>> print(eng2sp)
{'one': 'uno', 'two': 'dos', 'three': 'tres'}
>>> print(eng2sp["two"])
dos
>>> print(eng2sp["four"])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'four'
```

Fungsi **len()** dapat digunakan pada dictionary untuk mengembalikan jumlah pasangan key-value. Operator **in** dapat digunakan untuk mengecek apakah suatu key ada dalam dictionary. Untuk mengecek keberadaan suatu value, dapat digunakan method **values()** yang mengembalikan semua value dalam dictionary. Perbedaan performa antara operator **in** pada list dan dictionary juga penting, pada list Python menggunakan algoritma pencarian linear ( $O(n)$ ),

sedangkan pada dictionary Python menggunakan algoritma Hash Table dengan waktu pencarian yang konstan tidak bergantung pada jumlah item ( $O(1)$ ).

```
>>> "one" in eng2sp
True
>>> "uno" in eng2sp
False
>>> vals = list(eng2sp.values())
>>> print(vals)
['uno', 'dos', 'tres']
>>> "uno" in vals
True
>>> len(eng2sp)
3
```

### Dictionary Sebagai Set Penghitung (*counters*)

Salah satu kegunaan dictionary yang sangat praktis adalah sebagai model penghitung atau counter. Sebagai contoh, apabila diberikan sebuah string dan diminta untuk menghitung berapa kali masing-masing karakter muncul, terdapat beberapa pendekatan yang bisa digunakan. Pendekatan pertama adalah membuat 26 variabel terpisah untuk setiap huruf alfabet. Pendekatan kedua adalah membuat list dengan 26 elemen dan menggunakan fungsi `ord()` untuk mengonversi karakter menjadi angka sebagai indeks. Pendekatan ketiga adalah menggunakan dictionary dengan karakter sebagai key dan jumlah kemunculan sebagai value. Keunggulan pendekatan ketiga adalah tidak perlu mengetahui terlebih dahulu karakter apa saja yang akan muncul.

```
1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     if c not in d:
5         d[c] = 1
6     else:
7         d[c] = d[c] + 1
8 print(d)
```

Model komputasi seperti di atas disebut dengan histogram, yang merupakan istilah statistika untuk menggambarkan frekuensi kemunculan suatu elemen. Perulangan `for` menelusuri setiap karakter dalam string, jika karakter belum ada sebagai key dalam dictionary maka item baru dibuat dengan value awal 1, dan jika sudah ada maka valuenya bertambah 1. Sebagai contoh untuk kata 'brontosaurus', histogram yang dihasilkan adalah:

```
{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}
```

Dictionary juga memiliki method **`get()`** yang dapat mempersingkat penulisan loop histogram. Method ini menerima dua argumen yaitu key yang dicari dan default value yang dikembalikan apabila key tidak ditemukan. Dengan memanfaatkan method `get()`, empat baris

statement if dapat diringkas menjadi satu baris saja. Penggunaan `get()` dalam loop penghitungan telah menjadi idiom yang sangat umum digunakan dalam Python karena lebih ringkas dengan hasil yang sama.

### Dictionary dan File

Kegunaan lain dari dictionary yang umum adalah untuk menghitung kemunculan kata-kata dalam sebuah file teks. Program akan membaca setiap baris dalam file, memecah setiap baris menjadi daftar kata menggunakan method `split()`, kemudian melalui perulangan bersarang (nested loop) akan menghitung setiap kata menggunakan dictionary. Outer loop bertugas membaca baris demi baris dari file, sementara inner loop mengiterasi setiap kata pada baris yang sedang diproses. Kombinasi nested loop ini memastikan bahwa ada proses perhitungan untuk setiap kata pada setiap baris file input.

Setiap kata yang ditemukan akan dicek keberadaannya dalam dictionary: jika belum ada, ditambahkan dengan value awal 1; jika sudah ada, valuenya bertambah 1 menggunakan operator singkat `+=` yang setara dengan penulisan `counts[word] = counts[word] + 1`. Operator serupa lainnya antara lain `-=` untuk pengurangan, `*=` untuk perkalian, dan `/=` untuk pembagian.

Sebagai contoh, teks yang digunakan adalah teks dari Romeo and Juliet:

```
But soft what light through yonder window breaks  
It is the east and Juliet is the sun  
Arise fair sun and kill the envious moon  
Who is already sick and pale with grief
```

Source codenya:

```
1  fname = input('Enter the file name: ')  
2  try:  
3      fhand = open(fname)  
4  except:  
5      print('File cannot be opened:', fname)  
6      exit()  
7  
8  counts = dict()  
9  for line in fhand:  
10     words = line.split()  
11     for word in words:  
12         if word not in counts:  
13             counts[word] = 1  
14         else:  
15             counts[word] += 1  
16  
17  print(counts)
```

Outputnya:

```
Enter the file name: romeo.txt
{'and': 3, 'envious': 1, 'already': 1, 'fair': 1,
'is': 3, 'through': 1, 'pale': 1, 'yonder': 1,
'what': 1, 'sun': 2, 'Who': 1, 'But': 1, 'moon': 1,
>window': 1, 'sick': 1, 'east': 1, 'breaks': 1,
'grief': 1, 'with': 1, 'light': 1, 'It': 1, 'Arise': 1,
'kill': 1, 'the': 3, 'soft': 1, 'Juliet': 1}
```

### Looping dan Dictionary

Saat dictionary digunakan dalam pernyataan for, Python akan menelusuri setiap key yang ada di dalamnya. Di dalam loop, key yang diperoleh dapat digunakan untuk mengakses value yang bersesuaian melalui operator indeks. Pola ini dapat dimanfaatkan untuk berbagai keperluan, misalnya untuk menemukan semua entri yang valuenya melebihi angka tertentu. Output yang ditampilkan memperlihatkan bahwa key tidak berada dalam pola urutan tertentu, namun hal ini tidak menjadi masalah karena akses tetap dilakukan melalui key.

```
1 counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2 for key in counts:
3     if counts[key] > 10 :
4         print(key, counts[key])
```

Apabila ingin mencetak key dalam urutan alfabet, langkah-langkahnya adalah yang pertama membuat list dari semua key menggunakan method `keys()`. Kedua, mengurutkan list tersebut menggunakan method `sort()`. Ketiga, melakukan iterasi melalui sorted list dan mencetak

```
1 counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2 lst = list(counts.keys())
3 print(lst)
4 lst.sort()
5 for key in lst:
6     print(key, counts[key])
```

setiap pasangan key-value. Dengan cara ini, meskipun dictionary sendiri tidak memiliki urutan yang pasti, isinya tetap dapat ditampilkan dalam urutan yang diinginkan.

### Advanced Text Parsing

Pada sub-bab sebelumnya, contoh file yang digunakan sudah dihilangkan tanda bacanya. Namun dalam situasi nyata, file teks biasanya masih mengandung tanda baca lengkap. Masalah yang muncul adalah bahwa method `split()` hanya memisahkan kata berdasarkan spasi, sehingga kata seperti 'soft!' dan 'soft' dianggap sebagai dua kata berbeda dan disimpan secara terpisah dalam

dictionary. Demikian pula perbedaan huruf kapital, misalnya 'Who' dan 'who', juga dihitung sebagai kata yang berbeda.

Untuk mengatasi permasalahan tersebut, Python menyediakan beberapa method string. Method `lower()` mengubah seluruh huruf menjadi huruf kecil. Method `translate()` dikombinasikan dengan `str.maketrans()` digunakan untuk menghapus karakter tertentu. Modul string bawaan Python menyediakan konstanta `string.punctuation` yang berisi semua karakter tanda baca standar. Implementasi yang lengkap menggabungkan `rstrip()` untuk membersihkan whitespace, `translate()` untuk menghapus tanda baca, `lower()` untuk menyamakan kapitalisasi, dan `split()` untuk memecah menjadi kata-kata. Dengan pendekatan ini, kata yang sama dalam variasi penulisan apa pun akan dihitung sebagai satu kata yang sama, menghasilkan perhitungan frekuensi yang lebih akurat.

```
1  import string
2
3  fname = input('Enter the file name: ')
4  try:
5      fhand = open(fname)
6  except:
7      print('File cannot be opened:', fname)
8      exit()
9
10 counts = dict()
11 for line in fhand:
12     line = line.rstrip()
13     line = line.translate(line.maketrans('', '',
14                                     string.punctuation))
15     line = line.lower()
16     words = line.split()
17     for word in words:
18         if word not in counts:
19             counts[word] = 1
20         else:
21             counts[word] += 1
22 print(counts)
```

Output dari programnya adalah:

```
Enter the file name: romeo-full.txt
{'swearst': 1, 'all': 6, 'afeard': 1, 'leave': 2, 'these': 2,
'kinsmen': 2, 'what': 11, 'thinkst': 1, 'love': 24, 'cloak': 1,
a': 24, 'orchard': 2, 'light': 5, 'lovers': 2, 'romeo': 40,
'maiden': 1, 'whiteupturned': 1, 'juliet': 32, 'gentleman': 1,
'it': 22, 'leans': 1, 'canst': 1, 'having': 1, ...}
```

## BAGIAN 2: LATIHAN MANDIRI

Link Github repository: [https://github.com/ifgodwillsit/71251221\\_febrian](https://github.com/ifgodwillsit/71251221_febrian)

### Latihan 12.1

```
1 dictionary = {}
2
3 n = int(input("Masukkan jumlah data: "))
4 for i in range(n):
5     key = int(input(f" Key ke-{i+1} : "))
6     value = int(input(f" Value ke-{i+1} : "))
7     dictionary[key] = value
8
9 print("\nDictionary :", dictionary)
10 print()
11 print(f"{'key':<10} {'value':<10} {'item':<10}")
12
13 item_num = 1
14 for key, value in dictionary.items():
15     print(f"{'key':<10} {'value':<10} {'item_num':<10}")
16     item_num += 1
```

Gambar 1.1: Source code Latihan 12.1

Konsep dari latihan ini adalah untuk mendapatkan nilai key, value, dan item dari sebuah dictionary. Ini kubuat konsepnya sebagai user input, sehingga line 4-7 itu berupa user input dan loopnnya ( $O(n)$  tergantung banyak key dari line 3). Line 9-11 mengeprint ulang dictionarynya, kenapa ada `:<10` di situ? `:<10` digunakan sebagai f-string formatting rata kiri dengan lebar 10 karakter. Nah, line 13-16 adalah pengambilan key, value, dan itemnya. Item\_num adalah iterasi dari item (key-value pair ini item ke berapa), kemudian karena dictionary berisi 2 anggota yakni key dan value maka kita bisa mengambil dua-duanya sekaligus dalam 1 for (menggunakan method `items()`).

```
Dictionary : {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}

key      value      item
1         10         1
2         20         2
3         30         3
4         40         4
5         50         5
6         60         6
```

Gambar 1.2: Output Latihan 12.1

## Latihan 12.2

```
1  lista = input("Masukkan List A: ").split(',')
2  listb = input("Masukkan List B: ").split(',')
3
4  lista = [x.strip() for x in lista]
5  listb = [x.strip() for x in listb]
6
7  dictionary = dict(zip(lista, listb))
8
9  print("\nDictionary :", dictionary)
```

Gambar 1.3: Source code Latihan 12.2

Konsep dari latihan ini adalah untuk memetakan atau menggabungkan dua list menjadi satu dictionary. Line 1-2 digunakan untuk user input listnya, di mana line 4-5 mengolah input itu menjadi list. Line 7 mengimplementasikan syntax zip untuk memasangkan elemen lista dan listb secara berurutan, lalu langsung mengkonversinya jadi dict. Line 9 print hasil dictionarynya.

```
Masukkan List A: red, green, blue
Masukkan List B: #FF0000, #008000, #0000FF

Dictionary : {'red': '#FF0000', 'green': '#008000', 'blue': '#0000FF'}
```

Gambar 1.4: Output Latihan 12.2

## Latihan 12.3

```
1  fname = input("Masukkan nama file : ")
2
3  try:
4      fhand = open(fname)
5  except FileNotFoundError:
6      print("File tidak bisa dibuka :", fname)
7      exit()
8
9  counts = dict()
10
11 for line in fhand:
12     if not line.startswith("From "):
13         continue
14     words = line.split()
15     email = words[1]
16     counts[email] = counts.get(email, 0) + 1
17
18 print(counts)
```

Gambar 1.5: Source code Latihan 12.3

Konsep dari latihan ini adalah menampilkan banyak email masuk dari suatu akun dan disajin dalam bentuk dictionary. Line 1-7 digunakan untuk user input nama file dan mengolah isi file supaya compiler bisa read, jika tidak maka print line 6. Variabel counts dibuat sebagai



dictionary kosong. Line 11-16 merupakan perulangan yang digunakan untuk mencari baris yang diawali From, kemudian mengolah supaya nama akun emailnya dapat diambil. Kemudian dimasukkan ke dalam dictionary counts. Jika nama email belum ada di dictionary maka valuenya secara default adalah 1, jika nama email itu sudah ada di dictionary maka valuenya akan bertambah 1.

```
Masukkan nama file : mbox-short.txt
{'stephen.marquard@uct.ac.za': 2, 'louis@media.berkeley.edu': 3, 'zqian@umich.edu': 4, 'rjlowe@iupui.edu': 2, 'cwen@iupui.edu': 5, 'gsilver@umich.edu': 3, 'wagnermr@iupui.edu': 1, 'antranig@caret.cam.ac.uk': 1, 'gopal.ramasammycook@gmail.com': 1, 'david.horwitz@uct.ac.za': 4, 'ray@media.berkeley.edu': 1}
```

Gambar 1.6: Output Latihan 12.3

#### Latihan 12.4

```
1  fname = input("Masukkan nama file : ")
2
3  try:
4      fhand = open(fname)
5  except FileNotFoundError:
6      print("File tidak bisa dibuka :", fname)
7      exit()
8
9  counts = dict()
10
11 for line in fhand:
12     if not line.startswith("From "):
13         continue
14     words = line.split()
15     email = words[1]
16     domain = email.split("@")[1]
17     counts[domain] = counts.get(domain, 0) + 1
18
19 print(counts)
```

Gambar 1.7: Source code Latihan 12.4

Konsep latihan ini adalah untuk menghitung jumlah pesan yang dikirim ke domain-domain yang ada dari suatu file. Seperti latihan 12.3, line 1-7 digunakan untuk mengakses file. Line 9 digunakan untuk membuat dictionary kosong. Line 11 merupakan perulangan yang digunakan untuk mencari setiap baris yang diawali dengan From (sisanya email), lalu mengambil hanya domain dari akun email yang didapat. Kemudian apa yang didapat itu dimasukkan ke dictionary, jika belum ada maka akan dimasukkan sebagai key baru dengan default value 1, jikalau sudah maka value bertambah 1.

```
Masukkan nama file : mbox-short.txt  
{'uct.ac.za': 6, 'media.berkeley.edu': 4, 'umich.edu': 7, 'iupui.edu': 8, 'caret.cam.ac.  
uk': 1, 'gmail.com': 1}
```

Gambar 1.8: Output Latihan 12.4